BÖLÜM

4

ComboBox ve ListBox ile Liste İşlemleri



BU BÖLÜMÜN SONUNDA

- Items koleksiyonunu ve sıfırdan başlayan indis mantığını açıklayabilecek,
- Add, AddRange ve döngü ile listelere veri ekleyebilecek,
- Clear, Remove ve RemoveAt ile veri silebilecek, SelectedIndex = −1 durumunu kontrol edebilecek,
- Silme sonrasında indislerin nasıl kaydığını izleyebilecek,
- SelectedIndexChanged olayıyla seçime tepki veren uygulamalar yazabileceksiniz.

KampüsCepte · öğrenciler için ücretsiz ders notları

Şehir listeleri, personel kayıtları, ürün sepetleri... Pek çok uygulamanın merkezinde bir liste vardır. Bu bölümde ComboBox ve ListBox kontrollerine veri eklemeyi, seçili veriyi bulmayı ve silmeyi öğreniyoruz.

4.1 Items Koleksiyonu ve İndis Mantığı

ComboBox ve ListBox'taki elemanlar **Items** adlı bir koleksiyonda tutulur. Koleksiyon bir dizi gibi davranır: her elemanın bir **indisi** (sıra numarası) vardır ve indisler **0'dan başlar**. Beş elemanlı bir listede indisler 0–4 arasındadır, eleman sayısı ise `Items.Count` ile öğrenilir.

0	1	2	3
Elazığ	Malatya	Ankara	İstanbul

Şekil 4.1 *Dört elemanlı bir liste. Count = 4, son elemanın indisi Count − 1 = 3.*

Üye	Görevi	Örnek
`Items.Add(x)`	Listenin sonuna bir eleman ekler	`listBox1.Items.Add("Van");`
`Items.AddRange(dizi)`	Bir dizinin tüm elemanlarını döngüye gerek kalmadan ekler	`comboBox1.Items.AddRange(ilce);`
`Items.Clear()`	Tüm elemanları siler	`comboBox1.Items.Clear();`
`Items.Remove(x)`	Değeri x olan **ilk** elemanı siler (yoksa bir şey yapmaz)	`listBox1.Items.Remove("Van");`
`Items.RemoveAt(i)`	i indisli elemanı siler	`listBox1.Items.RemoveAt(0);`
`Items.Count`	Eleman sayısı	`label1.Text = listBox1.Items.Count + " kayıt";`
`SelectedIndex`	Seçili elemanın indisi; **seçim yoksa −1**	`int i = listBox1.SelectedIndex;`
`SelectedItem`	Seçili elemanın kendisi; seçim yoksa boş (null)	`listBox1.SelectedItem.ToString()`

DİKKAT

`RemoveAt` metoduna geçersiz bir indis verilirse (negatif ya da `Count` 'a eşit veya büyük) program **ArgumentOutOfRangeException** hatası verir. En sık görülen iki durum şunlardır: kullanıcı hiçbir şey seçmeden "Seçili Sil"e basar (`RemoveAt(-1)`) ya da boş listede "Sil"e basar (`RemoveAt(0)`). Silmeden önce `SelectedIndex != -1` ve `Items.Count > 0` kontrolleri yapılmalıdır.

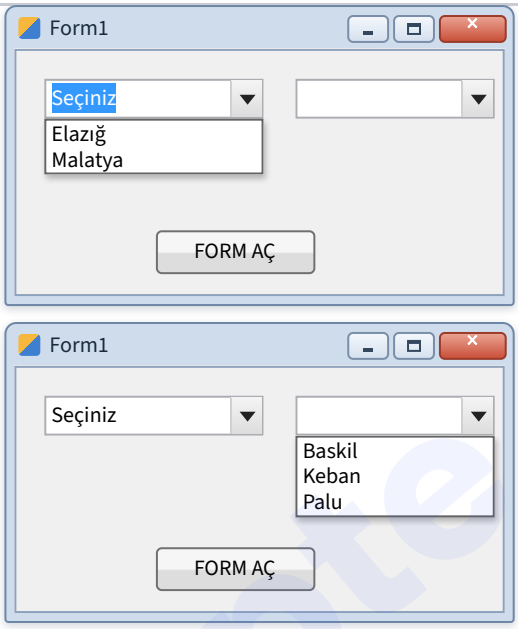
4.2 ComboBox'a Veri Ekleme

Listeler genellikle form açılırken, yani **Form1_Load** olayında doldurulur. Üç yol vardır. **1. yol:** `Add` ile elemanlar tek tek eklenir. **2. yol:** önce bir dizi oluşturulur, sonra `AddRange` ile dizinin tüm elemanları döngüye gerek kalmadan eklenir.

KampüsCepte · öğrenciler için ücretsiz ders notları

```
private void Form1_Load(object sender, EventArgs
e)
{
    // 1. yol: Add ile tek tek ekleme
    comboBox1.Text = "Seçiniz";
    comboBox1.Items.Add("Elazığ");
    comboBox1.Items.Add("Malatya");

    // 2. yol: dizi oluşturulur, AddRange ile
topluca eklenir
    string[] ilce = { "Baskil", "Keban", "Palu"
};
    comboBox2.Items.AddRange(ilce);
}
```



NOT

Sunumdaki örnekte ilçe dizisinde *harput* de yer almaktadır. Harput, Elazığ'ın bir ilçesi değil, merkeze bağlı tarihî bir yerleşim yeridir. Bu nedenle örnekte Elazığ'ın ilçelerinden **Palu** kullanılmıştır.

3. yol: dizinin elemanları bir döngüyle tek tek `Add` edilir. Döngü sınırı olarak elle 4 yazmak yerine `sehir.Length` kullanmak, diziye eleman eklendiğinde kodu değiştirme ihtiyacını ortadan kaldırır.

4.3 ComboBox'tan Veri Silme

Tümünü silme: `Items.Clear()` komutu ile listedeki tüm veriler silinir.

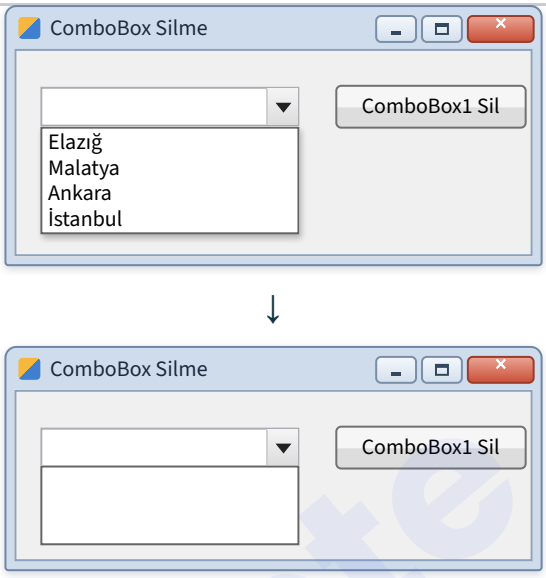


Aradan bir eleman çekildiğinde arkadakiler bir sıra öne kayar

KampüsCepte · öğrenciler için ücretsiz ders notları

```
private void Form1_Load(object sender, EventArgs
e)
{
    // 3. yol: dizi elemanları döngüyle tek tek
eklenir
    string[] sehir = { "Elazığ", "Malatya",
"Ankara", "İstanbul" };
    for (int i = 0; i < sehir.Length; i++)
    {
        comboBox1.Items.Add(sehir[i]);
    }
}

private void button1_Click(object sender,
EventArgs e)
{
    comboBox1.Items.Clear();   // tüm elemanları
siler
}
```



Seçili veriyi silme: Önce ComboBox'ta seçili olan verinin indisi `SelectedIndex` ile öğrenilir, sonra `RemoveAt` ile o indisteki veri silinir. Seçim yapılmamışsa indis –1 olacağından silmeden önce kontrol edilir.

```
private void button1_Click(object sender,
EventArgs e)
{
    int indis = comboBox1.SelectedIndex;   //
seçili elemanın indisini alır
    if (indis != -1)
    {
        comboBox1.Items.RemoveAt(indis);   // o
indisteki elemanı siler
    }
    else
    {
        MessageBox.Show("Önce silinecek elemanı
seçiniz.");
    }
}
```



Seçili “Ankara” verisi (indis 2) silindikten sonra kendisinden sonraki “İstanbul” bir sıra yukarı kayar ve indisi 3'ten 2'ye düşer. Bu **kayma** davranışı, silme işlemlerinin döngüyle yapıldığı durumlarda çok önemlidir:

Silmeden önce

0	1	2	3
Elazığ	Malatya	~~Ankara~~	İstanbul

RemoveAt(2) sonrası

0	1	2
Elazığ	Malatya	İstanbul

Şekil 4.2 *Bir eleman silindiğinde sonraki tüm elemanların indisi bir azalır.*

KampüsCepte · öğrenciler için ücretsiz ders notları

4.4 ListBox'a Veri Ekleme

ARABA MARKA yazan bir Label, bir TextBox, **ekle** butonu ve bir ListBox içeren form oluşturulur. Butona her tıklandığında kutudaki marka listeye eklenir.

SUNUMDAKİ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs e)
{
    string marka;
    marka = textBox1.Text;
    listBox1.Items.Add(marka);
}
```



GELİŞTİRİLMİŞ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs e)
{
    string marka = textBox1.Text.Trim();   //
baştaki/sondaki boşluklar atılır
    if (marka == "")
    {
        MessageBox.Show("Marka adı boş bırakılamaz.");
        return;
    }
    listBox1.Items.Add(marka);
    textBox1.Clear();                      // kutu
temizlenir
    textBox1.Focus();                      // imleç
yeniden kutuya gelir
}
```

Sunumdaki çözüm, kutu boşken bile listeye boş bir satır ekler ve her eklemeden sonra kutunun elle temizlenmesini gerektirir. Geliştirilmiş çözümde `Trim()` baştaki ve sondaki boşlukları atar, boş giriş engellenir, `Clear()` kutuyu boşaltır ve `Focus()` imleci yeniden kutuya getirir. Kullanıcı Tab ya da fareye ihtiyaç duymadan art arda veri girebilir.

4.5 ListBox'tan Veri Silme

Forma **sil** ve **seçili sil** butonları eklenir. **sil** butonu listenin başından itibaren sırayla siler (`RemoveAt(0)`), **seçili sil** ise kullanıcının seçtiği satırı siler.

KampüsCepte · öğrenciler için ücretsiz ders notları

```
private void button2_Click(object sender, EventArgs e)
// "sil"
{
    if (listBox1.Items.Count > 0)
    {
        listBox1.Items.RemoveAt(0);         // listenin
başından itibaren siler
    }
}

private void button3_Click(object sender, EventArgs e)
// "seçili sil"
{
    if (listBox1.SelectedIndex != -1)
    {

listBox1.Items.RemoveAt(listBox1.SelectedIndex);   //
seçilen satırı siler
    }
}
```



İPUCU

Listede çoklu seçim açıksa (`SelectionMode = MultiExtended`) `SelectedIndex` yalnızca seçilenlerden birini verir. Tüm seçili satırlar `SelectedItems` koleksiyonunda bulunur.

4.6 Sütunlu Kayıt Listesi

Dışarıdan ad, soyad, doğum yeri ve meslek bilgileri girilir. **KAYIT EKLE** butonuna tıklanınca bu bilgiler ListBox'a tek satır olarak eklenir, **KAYIT SİL** butonuna tıklanınca eklenmiş veriler satır satır silinir. Değerler arasına konan `"\t"` (sekme karakteri), ListBox'ta sütunların hizalı görünmesini sağlar. ListBox'ın `UseTabStops` özelliği varsayılan olarak açıktır.

```
private void button1_Click(object sender,
EventArgs e)   // KAYIT EKLE
{
    listBox1.Items.Add(textBox1.Text + "\t" +
textBox2.Text + "\t"
                + textBox3.Text + "\t" +
textBox4.Text);
}

private void button2_Click(object sender,
EventArgs e)   // KAYIT SİL
{
    if (listBox1.Items.Count > 0)
    {
        listBox1.Items.RemoveAt(0);
    }
}
```



KampüsCepte · öğrenciler için ücretsiz ders notları

Sunumdaki çözümde `RemoveAt(0)` satırı, liste boşken basıldığında hata vereceği için `Count > 0` kontrolü eklenmiştir.

4.7 SelectedIndexChanged Olayı ve Personel Uygulaması

ComboBox'ta seçili eleman her değiştiğinde **SelectedIndexChanged** olayı tetiklenir. Bu olay sayesinde kullanıcı bir butona basmadan, yalnızca seçim yaparak işlem başlatabilir. Aşağıdaki uygulamada AD ve SOYAD kutularına girilen veriler EKLE butonuyla PERSONELLER listesine aktarılır. Listedeki veriler seçili olarak ya da sırayla silinebilir. ARKA PLAN RENGİ listesinden bir renk seçildiğinde formun arka planı o renge dönüşür.



Şekil 4.3 *Personel uygulaması: listeden mavi seçildiğinde form arka planı maviye döner.*

PERSONEL UYGULAMASININ TÜM KODU

```
private void Form1_Load(object sender, EventArgs e)
{
    string[] dizi = { "kırmızı", "beyaz", "mavi" };
    comboBox1.Items.AddRange(dizi);
}

private void button1_Click(object sender, EventArgs e)   // EKLE
{
    listBox1.Items.Add(textBox1.Text + "\t" + textBox2.Text);
}

private void button2_Click(object sender, EventArgs e)   // SEÇİLİ SİL
{
    int sec = listBox1.SelectedIndex;          // seçili indis
    if (sec != -1)
    {
        listBox1.Items.RemoveAt(sec);          // o indisi siler
    }
}

private void button3_Click(object sender, EventArgs e)   // SIRAYLA SİL
{
    if (listBox1.Items.Count > 0)
    {
        listBox1.Items.RemoveAt(0);
    }
}
```


KampüsCepte · öğrenciler için ücretsiz ders notları

```
private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    // this: kodun içinde bulunduğu Form1 nesnesini ifade eder
    if (comboBox1.SelectedIndex == 0)
    {
        this.BackColor = Color.Red;
    }
    else if (comboBox1.SelectedIndex == 1)
    {
        this.BackColor = Color.White;
    }
    else if (comboBox1.SelectedIndex == 2)
    {
        this.BackColor = Color.Blue;
    }
}
```

NOT

Sunumdaki çözümde *beyaz* seçeneği (indis 1) için `Color.Yellow` yazılmıştır. Bu nedenle beyaz seçildiğinde form sarı olur. Doğrusu `Color.White`'tır. Ayrıca üç ayrı `if` yerine `else if` zinciri kullanılmıştır. Bir seçim yapıldığında diğer şartların boşuna kontrol edilmesine gerek yoktur.

NESNE TABANLI BAKIŞ

Renk uygulamasında kullanılan `this` anahtar kelimesi, kodun içinde bulunduğu **nesneyi**, yani çalışan Form1 penceresini ifade eder. `this.BackColor = Color.Red;` ifadesi "bu formun arka plan rengini kırmızı yap" demektir. Aynı Form1 sınıfından iki pencere açılsaydı, her pencerenin `this`'i kendisi olurdu ve biri değiştiğinde diğeri etkilenmezdi. Her nesnenin özellik değerleri kendine aittir.

ÖZET

- ComboBox ve ListBox elemanları **Items** koleksiyonunda tutulur. İndisler **0**'dan başlar, son indis `Count - 1`'dir.
- Ekleme `Add`, `AddRange` ya da döngüyle yapılır. Silme işlemleri `Clear` (tümü), `Remove` (değere göre ilk eşleşen) ve `RemoveAt` (indise göre) ile yapılır.
- Seçim yoksa `SelectedIndex = -1` olur. Geçersiz indisle `RemoveAt` çağrılırsa **ArgumentOutOfRangeException** oluşur.
- Bir eleman silindiğinde ondan sonraki elemanların indisi bir azalır. Döngüyle silerken bu kayma hesaba katılmalıdır.
- `"\t"` ile ListBox satırlarında sütun hizası sağlanır.
- Seçim değişince **SelectedIndexChanged** olayı tetiklenir. `this`, kodun ait olduğu form nesnesidir.

KampüsCepte · öğrenciler için ücretsiz ders notları